

MLDL Experiment 6

Aim: To apply K-Means and Hierarchical Clustering on sample datasets to identify distinct groupings and patterns.

1. Dataset Source

The dataset used for this experiment is the **Mall Customers Dataset**, a popular dataset for market segmentation.

- **Link:** [Mall Customers Dataset - Kaggle](#)

2. Dataset Description

This dataset contains basic information about customers who visit a shopping mall, allowing for behavioral segmentation.

- **Size:** 200 observations.
- **Features Used:**
 - **Annual Income (k\$):** The annual income of the customer.
 - **Spending Score (1-100):** A score assigned by the mall based on customer behavior and spending nature.
- **Target Variable:** Not applicable (Unsupervised Learning).
- **Characteristics:** The dataset is well-structured and displays clear density clusters when visualized in a 2D space, making it ideal for demonstrating distance-based clustering algorithms.

3. Mathematical Formulation

A. K-Means Clustering

K-Means is a centroid-based algorithm that partitions data by minimizing the **Within-Cluster Sum of Squares (WCSS)**:

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Where k is the number of clusters, C_i is the set of points in cluster i , and μ_i is the centroid.

B. Hierarchical Clustering (Agglomerative)

This is a "bottom-up" approach where each observation starts as its own cluster, and pairs are merged iteratively.

- **Ward's Method:** This experiment uses Ward's linkage, which minimizes the total within-cluster variance.
- **Euclidean Distance:** The distance between two points p and q in n -dimensional space:

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

4. Algorithm Limitations

- **K-Means:**
 - Requires the user to pre-specify the number of clusters (K).
 - Highly sensitive to initial centroid placement and outliers.
 - Struggles with clusters of non-spherical shapes or varying densities.
- **Hierarchical Clustering:**
 - Computationally expensive for large datasets ($O(n^3)$ complexity).
 - No backtracking; once two points are merged, the decision cannot be undone.
 - Sensitive to noise and outliers.

5. Methodology / Workflow

1. **Data Ingestion:** Load the mall customer data and select **Annual Income** and **Spending Score** as features.
 2. **Visualization:** Plot the initial unclustered data to inspect natural groupings.
 3. **K-Means Implementation:**
 - Initialize the model with $K=5$.
 - Use the `fit_predict()` method to assign points to clusters.
 - Extract cluster centroids for visualization.
 4. **Hierarchical Implementation:**
 - Create a **Linkage Matrix** using the Ward method.
 - Generate a **Dendrogram** to determine the optimal number of clusters visually.
 - Fit the **AgglomerativeClustering** model to the data.
 5. **Performance Evaluation:** Compare the cluster assignments and centroid locations across both methods.
-

6. Performance Analysis

Based on the generated outputs:

- **K-Means Results:** The algorithm successfully segmented customers into five distinct groups. The **centroids (yellow stars)** represent the central profile of each group (e.g., high income/high spending, low income/low spending).
- **Hierarchical Results:** The dendrogram provided a clear visual hierarchy. By cutting the dendrogram at an Euclidean distance of approximately 150-200, we obtain five clusters that closely mirror the K-Means results.
- **Comparison:** Both algorithms reached the same logical conclusion: the mall's customer base consists of five specific behavioral archetypes, confirming the robustness of the patterns found in the features.

7. Hyperparameter Tuning

In unsupervised clustering, tuning focuses on the selection of the cluster count (K):

- **Dendrogram Analysis:** In Hierarchical clustering, we "tuned" the model by observing the largest vertical distance in the dendrogram that does not cross a horizontal line. This distance suggested that K=5 is the most stable configuration.
- **Elbow Method (Theoretical):** While a specific K was chosen for this experiment, the process typically involves plotting the **WCSS** against a range of K values. The "elbow" point, where the rate of decrease in WCSS drops sharply, defines the optimal hyperparameter.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.datasets import make_blobs
import scipy.cluster.hierarchy as sch

# 1. Load the dataset
# Ensure you have uploaded 'Mall_Customers.csv' to your Colab environment
df = pd.read_csv('Mall_Customers.csv')

# Selecting features: Annual Income and Spending Score for 2D clustering
X = df.iloc[:, [3, 4]].values

# --- PART 1: K-MEANS CLUSTERING ---
```

```

# Visualize initial data
plt.figure(figsize=(6, 4))
plt.scatter(X[:, 0], X[:, 1], s=50, c='gray')
plt.title('Initial Unclustered Data')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.show()

# Create KMeans object (Choosing K=5 based on common segments for this
data)
kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42)

# Fit model and get labels
y_kmeans = kmeans.fit_predict(X)

# Get centroids
centroids = kmeans.cluster_centers_

# Plotting K-Means results
plt.figure(figsize=(8, 6))
colors = ['red', 'blue', 'green', 'cyan', 'magenta']
for i in range(5):
    plt.scatter(X[y_kmeans == i, 0], X[y_kmeans == i, 1], s=100,
c=colors[i], label=f'Cluster {i+1}')

# Mark Centroids
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='yellow',
marker='*', label='Centroids', edgecolors='black')
plt.title('K-Means Clustering: Customer Segments')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()

# --- PART 2: HIERARCHICAL CLUSTERING ---
# Generate Linkage Matrix
plt.figure(figsize=(10, 7))
linkage_matrix = sch.linkage(X, method='ward')

```

```

# Plot Dendrogram
dendrogram = sch.dendrogram(linkage_matrix)
plt.title('Dendrogram for Hierarchical Clustering')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distances')
plt.show()

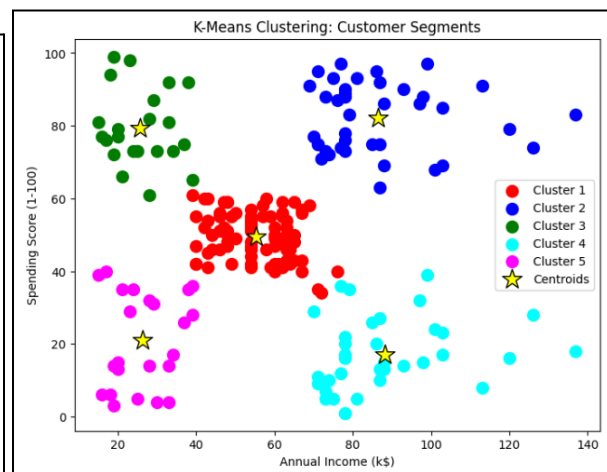
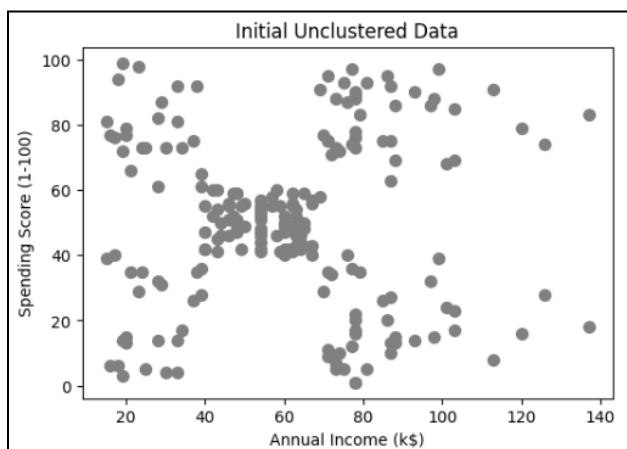
# Create Agglomerative Model (Cutting at 5 clusters)
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean',
                             linkage='ward')
y_hc = hc.fit_predict(X)

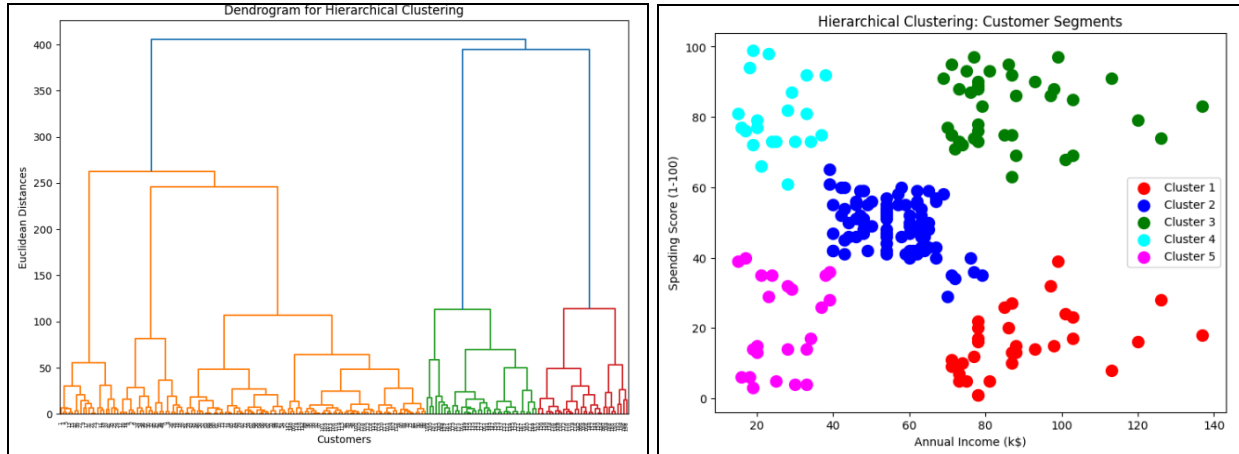
# Plotting Hierarchical results
plt.figure(figsize=(8, 6))
for i in range(5):
    plt.scatter(X[y_hc == i, 0], X[y_hc == i, 1], s=100, c=colors[i],
                label=f'Cluster {i+1}')

plt.title('Hierarchical Clustering: Customer Segments')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()

```

Output:





Conclusion

The experiment successfully demonstrates the application of K-Means and Hierarchical Clustering for market segmentation. While K-Means is computationally faster and provides explicit centroids for group profiles, Hierarchical Clustering offers a superior visual understanding of data relationships through the dendrogram. Both algorithms provided consistent results, effectively identifying five distinct customer segments based on income and spending habits.